MAGNATE ACADEMY

TECHNICAL ROUND 1 – AGENTIC AI

Questions and Answers

Part 1: Python Coding and Theory
Part 2: Agentic AI Theory
Part 3: Practical AI Agent Workflow

Prepared as a study and submission reference

PART 1 – PYTHON | 20 MARKS

A. Python Coding | 15 Marks

1. Count the Frequency of Each Character (3 marks)

Question: Write a Python program to count the frequency of each character in a given string.
Answer: Use a dictionary and increase each character's count.

```
text = input("Enter a string: ")
frequency = {}
for char in text:
    if char in frequency:
        frequency[char] += 1
    else:
        frequency[char] = 1
print(frequency)
```

Example input: programming
Output: {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}

2. Second Largest Distinct Number (3 marks)

Question: Find the second-largest distinct number without using sort() or sorted().

```
numbers = [10, 5, 8, 20, 20, 15]
largest = second = None
for num in numbers:
    if largest is None or num > largest:
        if num != largest:
            second = largest
        largest = num
    elif num != largest and (second is None or num > second):
        second = num
print(second if second is not None else "No second-largest distinct number")
```

Output: 15. The code tracks the two largest distinct values in one pass.

3. Remove Duplicate Elements (3 marks)

Question: Remove duplicates without changing the original order. Do not use set().

```
items = [1, 2, 3, 2, 4, 1, 5]
unique_items = []
for item in items:
    if item not in unique_items:
        unique_items.append(item)
print(unique_items)
```

Output: [1, 2, 3, 4, 5]

4. Two Sum (3 marks)

Question: Return the indices of two numbers that add up to the target. Do not use the same element twice.

```
def two_sum(nums, target):
    seen = {}
    for index, num in enumerate(nums):
        needed = target - num
        if needed in seen:
            return [seen[needed], index]
        seen[num] = index
    return []

print(two_sum([2, 7, 11, 15], 9))
```

Output: [0, 1]. A dictionary stores previously seen values and their indices.

5. Valid Palindrome (3 marks)

Question: Check whether a string reads the same forwards and backwards after lowercasing and removing non-alphanumeric characters.

```
def is_palindrome(s):
    cleaned = ""
    for char in s.lower():
        if char.isalnum():
            cleaned += char
    return cleaned == cleaned[::-1]
```

```
print(is_palindrome("A man, a plan, a canal: Panama"))  # True
print(is_palindrome("race a car"))                      # False
```

B. Python Theory | 5 Marks

1. What is the difference between a list and a tuple? Give an example.

List: Ordered and mutable; its contents can be changed. Example: `items = [1, 2, 3]`.
Tuple: Ordered and immutable; its contents cannot be changed after creation. Example: `point = (1, 2, 3)`.

2. What is the difference between mutable and immutable data types? Give two examples of each.

Mutable: Can be changed after creation. Examples: `list`, `dict`.
Immutable: Cannot be changed after creation. Examples: `int`, `str`.

3. What is exception handling? Explain try, except and finally with an example.

Exception handling lets a program handle runtime errors. `try` contains code that may fail; `except` handles an error; `finally` runs whether an error occurs or not.

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except (ValueError, ZeroDivisionError) as error:
    print("Invalid input:", error)
finally:
    print("Execution completed.")
```

4. What is list comprehension?

A concise way to create a list from an iterable, optionally applying a condition. Example: `squares = [x*x for x in range(1, 6)]` produces `[1, 4, 9, 16, 25]`.

5. What is a lambda function? How does it differ from a normal function?

A lambda is an anonymous, single-expression function. A normal function uses `def` and can contain multiple statements.

```
square = lambda x: x * x
print(square(5))  # 25

def square_normal(x):
    return x * x
```

PART 2 – AGENTIC AI | 10 MARKS

1. What are the advantages and limitations of reactive agents?

Advantages: Fast, simple, low-cost, and useful when actions can be selected directly from current input.
Limitations: Usually have little memory, limited planning, weak adaptation to unfamiliar situations, and may not learn from past events.

2. What is the Transformer architecture, and why is it important in modern LLMs?

A Transformer is a neural-network architecture that uses attention, especially self-attention, to learn relationships between tokens. It models context effectively and supports parallel processing during training. Transformers underpin many modern large language models.

3. What is prompt engineering, and why is it important when building AI agents?

Prompt engineering means designing clear instructions, context, constraints, and output formats for a model. It helps agents understand their role, select tools appropriately, and produce more consistent, relevant responses.

4. What is the difference between a system prompt and a user prompt?

System prompt: Defines the agent's role, behaviour, and constraints, e.g. “You are a helpful Python tutor.”
User prompt: Gives the specific request, e.g. “Explain Python dictionaries.”

5. When would you choose a multi-agent architecture instead of a single-agent architecture?

Use multiple agents when a task has distinct subtasks requiring different expertise, parallel work, or independent review. For example, research, analysis, and report writing can be handled by separate agents. A single agent is usually simpler for small tasks.

6. What is the difference between role, goal and backstory in CrewAI?

Role: The agent's function, such as Research Analyst.
Goal: The outcome it should achieve.
Backstory: Context about its experience or perspective that guides its behaviour.

7. What is the purpose of ChatPromptTemplate?

In LangChain, `ChatPromptTemplate` creates structured chat prompts from reusable message templates. It supports roles such as system and human and fills placeholders with runtime values.

8. What are the required parameters when creating a CrewAI Agent?

The commonly essential parameters are `role`, `goal`, and `backstory`. Parameters such as `llm`, `tools`, `verbose`, and `max_iter` are also configurable. Check the installed CrewAI version for exact requirements.

9. What is the default value of max_iter in CrewAI?

Commonly used CrewAI versions have used a default of **20** for `max_iter`. Defaults can change between versions, so verify the installed version's documentation.

10. What is the default value of n in the LangChain OpenAI completion model?

The traditional LangChain OpenAI completion wrapper commonly defaults n to **1**, meaning one completion is requested. Exact defaults may vary by wrapper and version.

PART 3 – AGENTIC AI | PRACTICAL (20 MARKS)

Task: Create an AI agent using LangChain or CrewAI that accepts a user query, uses an appropriate tool, and returns a meaningful response. Include the complete project code and push it to GitHub.

Sample solution: LangChain + local Ollama model + calculator tool

This example uses a local Ollama model, so a hosted API key is not required. The selected model must support tool calling for this agent interface to work.

```
# main.py
import os
from dotenv import load_dotenv
from langchain_ollama import ChatOllama
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain.agents import create_tool_calling_agent, AgentExecutor

load_dotenv()

@tool
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression using numbers and + - * / ( )."""
    allowed = set("0123456789+-*/(). ")
    if not expression or any(ch not in allowed for ch in expression):
        return "Invalid expression. Use basic arithmetic characters only."
    try:
        # For production, use a dedicated arithmetic parser instead.
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception:
        return "Could not evaluate that expression."

llm = ChatOllama(
    model=os.getenv("OLLAMA_MODEL", "qwen2.5:3b"),
    temperature=0
)
tools = [calculator]
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Use the calculator tool for arithmetic. "
               "Answer clearly and do not invent results if a tool fails."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])
agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(
    agent=agent, tools=tools, verbose=True,
    max_iterations=5, handle_parsing_errors=True
)

if __name__ == "__main__":
```

```
    question = input("Enter your question: ")
    result = agent_executor.invoke({"input": question})
    print("\nAgent response:", result["output"])
```

Security note: the calculator restricts characters and disables Python built-ins, but a dedicated arithmetic parser is safer for production. Do not expose general-purpose evaluation to untrusted input.

1. requirements.txt

```
langchain
langchain-ollama
python-dotenv
```

2. .env

```
OLLAMA_MODEL=qwen2.5:3b
```

This example has no API key. Do not commit real secrets or the .env file.

3. .gitignore

```
.env
.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.DS_Store
```

4. README.md (suggested contents)

```
# LangChain Calculator Agent

A command-line AI agent using LangChain and a local Ollama model.
It accepts questions, uses a calculator tool for arithmetic, and
returns a natural-language response.

## Prerequisites
- Python version supported by installed packages
- Ollama installed and running

## Setup
1. Create and activate a virtual environment.
2. Install dependencies: pip install -r requirements.txt
3. Pull the model: ollama pull qwen2.5:3b
4. Run: python main.py

## Example
Input: What is (25 * 4) + 10?
Expected answer: 110

## Files
- main.py: agent and calculator tool
- requirements.txt: dependencies
- .env: local model setting (not committed)
- .gitignore: excludes local environment files
```

5. Run and test

Run `ollama pull qwen2.5:3b`, install the requirements, ensure Ollama is running, then execute `python main.py`. Test a calculation and a general question. If the installed package version or model does not support tool calling, use compatible versions or a different supported agent pattern.

6. Push to GitHub

```
git init
git add main.py requirements.txt README.md .gitignore
git commit -m "Add LangChain calculator agent"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
git push -u origin main
```

Create an empty GitHub repository first, replace the placeholder remote URL with your repository URL, and run these commands in the project folder. Keep .env out of Git.

Submission checklist

• All five Python coding questions include code and examples.

• Python theory and Agentic AI theory questions are answered.

- Practical project includes main.py, requirements.txt, README.md, and .gitignore.
- Test the project locally before uploading.
- Confirm .env and secrets are excluded from Git.
- Submit the actual GitHub repository URL.